Implementation of DDA and Bresenham Line Drawing Algorithm.py

```python
from OpenGL.GL import *
from OpenGL.GLUT import *
from OpenGL.GLU import *
import numpy as np

# DDA Algorithm
def DDA(x1, y1, x2, y2):
    points = []
    dx = x2 - x1
    dy = y2 - y1

    if dx == 0:
        for y in range(y1, y2 + 1):
            points.append((x1, y))
        return points

    m = dy / dx
    y = y1

    if abs(dx) >= abs(dy):
        for x in range(x1, x2 + 1):
            points.append((round(x), round(y)))
            y += m
    else:
        x = x1
        step = 1 / m
        for y in np.arange(y1, y2 + 1, 1):
            points.append((round(x), round(y)))
            x += step
    return points


# Bresenham Algorithm (adjusted for all directions)
def Bresenham(x0, y0, x1, y1):
    points = []

    dx = abs(x1 - x0)
    dy = abs(y1 - y0)

sx = 1 if x0 < x1 else -1   # determine step direction for x
sy = 1 if y0 < y1 else -1   # determine step direction for y
```

```python
    err = dx - dy

    while True:
        points.append((x0, y0))

        if x0 == x1 and y0 == y1:
            break

        e2 = 2 * err

        if e2 > -dy:
            err -= dy
            x0 += sx

        if e2 < dx:
            err += dx
            y0 += sy

    return points


def show():
    glClear(GL_COLOR_BUFFER_BIT)          # Clear window

    glColor3f(0.0, 0.0, 0.0)              # Set drawing color (black)
    glBegin(GL_POINTS)                    # Start drawing points

    # Line 1 using DDA
    points1 = DDA(100,100,300,300)
    for x,y in points1:
        glVertex2i(x,y)

    # Line 2 using Bresenham (intersecting line)
    points2 = Bresenham(300,100,100,300)
    for x,y in points2:
        glVertex2i(x,y)

    glEnd()                               # End drawing
    glutSwapBuffers()                     # Refresh window


# Main setup
glutInit()                                # Initialize GLUT
glutInitDisplayMode(GLUT_RGBA | GLUT_DOUBLE)  # Set display mode
glutInitWindowSize(500, 500)              # Set window size
glutCreateWindow(b"Kamrul_43")            # Set window title (Name_Roll)

glClearColor(1.0, 1.0, 1.0, 1.0)          # Set background color (white)
gluOrtho2D(0, 500, 0, 500)                # Define 2D coordinate system

glutDisplayFunc(show)                     # Register display function
glutMainLoop()                            # Start main loop
```

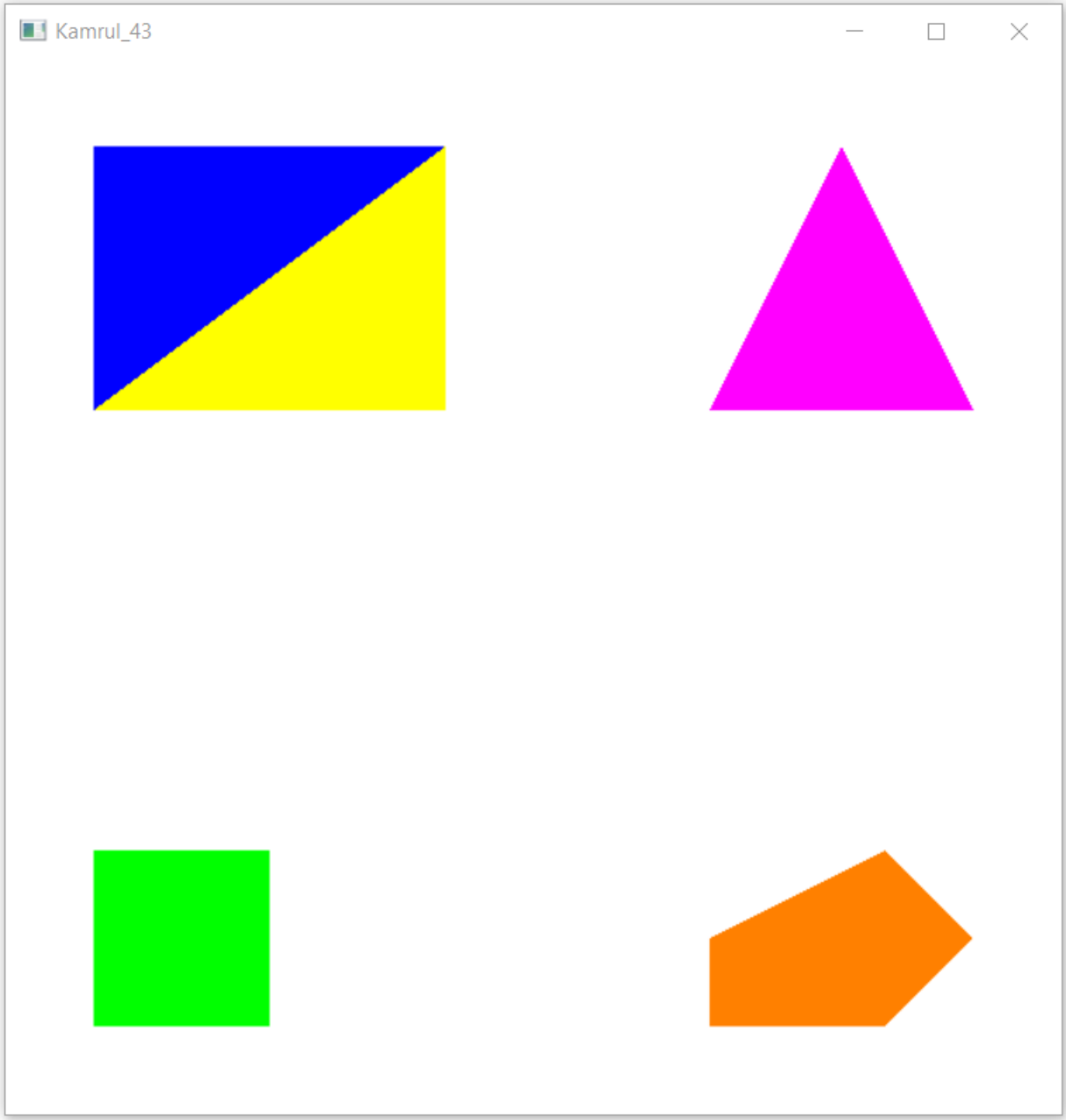
OUTPUT

Draw multiple non-overlapping objects in specific positions using OpenGL.py

```python
from OpenGL.GL import *
from OpenGL.GLUT import *
from OpenGL.GLU import *

def show():
    glClear(GL_COLOR_BUFFER_BIT)   # Clear window

    # Bottom-left: small green square
    glColor3f(0.0, 1.0, 0.0)       # Green color
    glBegin(GL_QUADS)
    glVertex2f(50, 50)
    glVertex2f(150, 50)
    glVertex2f(150, 150)
    glVertex2f(50, 150)
    glEnd()

    # Top-right: large purple triangle
    glColor3f(1.0, 0.0, 1.0)       # Purple color
    glBegin(GL_TRIANGLES)
    glVertex2f(400, 400)
    glVertex2f(550, 400)
    glVertex2f(475, 550)
    glEnd()

    # Top-left: rectangle using two triangles (different colors)
    glBegin(GL_TRIANGLES)

    # First triangle (Yellow)
    glColor3f(1.0, 1.0, 0.0)
    glVertex2f(50, 400)
    glVertex2f(250, 400)
    glVertex2f(250, 550)

    # Second triangle (Blue)
    glColor3f(0.0, 0.0, 1.0)
    glVertex2f(50, 400)
    glVertex2f(50, 550)
    glVertex2f(250, 550)

    glEnd()
```

```
    # Bottom-right: small orange pentagon
    glColor3f(1.0, 0.5, 0.0)       # Orange color
    glBegin(GL_POLYGON)
    glVertex2f(400, 50)
    glVertex2f(500, 50)
    glVertex2f(550, 100)
    glVertex2f(500, 150)
    glVertex2f(400, 100)
    glEnd()

    glutSwapBuffers()              # Refresh window


# Main setup
glutInit()                                      # Initialize GLUT
glutInitDisplayMode(GLUT_RGBA | GLUT_DOUBLE)
glutInitWindowSize(600, 600)                    # Window size
glutCreateWindow(b"Kamrul_43")                  # Window title

glClearColor(1.0, 1.0, 1.0, 1.0)                # White background
gluOrtho2D(0, 600, 0, 600)                     # 2D coordinate system

glutDisplayFunc(show)                           # Display function
glutMainLoop()                                 # Start program
```

OUTPUT



Draw two intersecting lines using DDA and Bresenham, show execution time difference.py

```python
from OpenGL.GL import *
from OpenGL.GLUT import *
from OpenGL.GLU import *
import time


# -------- Draw Text on Screen --------
def drawText(x, y, text):
    glRasterPos2f(x, y)   # set position for text
    for ch in text:
        glutBitmapCharacter(GLUT_BITMAP_HELVETICA_18, ord(ch))  # draw character


# -------- DDA Line Drawing --------
def DDA(x1, y1, x2, y2):
    points = []  # store points

    dx = x2 - x1   # difference in x
    dy = y2 - y1   # difference in y

    steps = int(max(abs(dx), abs(dy)))   # total steps

    x_inc = dx / steps   # x increment
    y_inc = dy / steps   # y increment

    x, y = x1, y1   # starting point

    for i in range(steps):
        points.append((round(x), round(y)))   # add point
        x += x_inc   # move x
        y += y_inc   # move y

    return points


# -------- Bresenham Line --------
def BresenhamLine(x0, y0, x1, y1):
    points = []

    dx = abs(x1 - x0)   # change in x
    dy = abs(y1 - y0)   # change in y

    sx = 1 if x0 < x1 else -1   # x direction
    sy = 1 if y0 < y1 else -1   # y direction

    err = dx - dy   # decision parameter

    while True:
        points.append((x0, y0))   # plot point

        if x0 == x1 and y0 == y1:
            break   # stop when reached end

        e2 = 2 * err   # calculate error

        if e2 > -dy:
            err -= dy
            x0 += sx   # move in x direction

        if e2 < dx:
            err += dx
            y0 += sy   # move in y direction

    return points
```

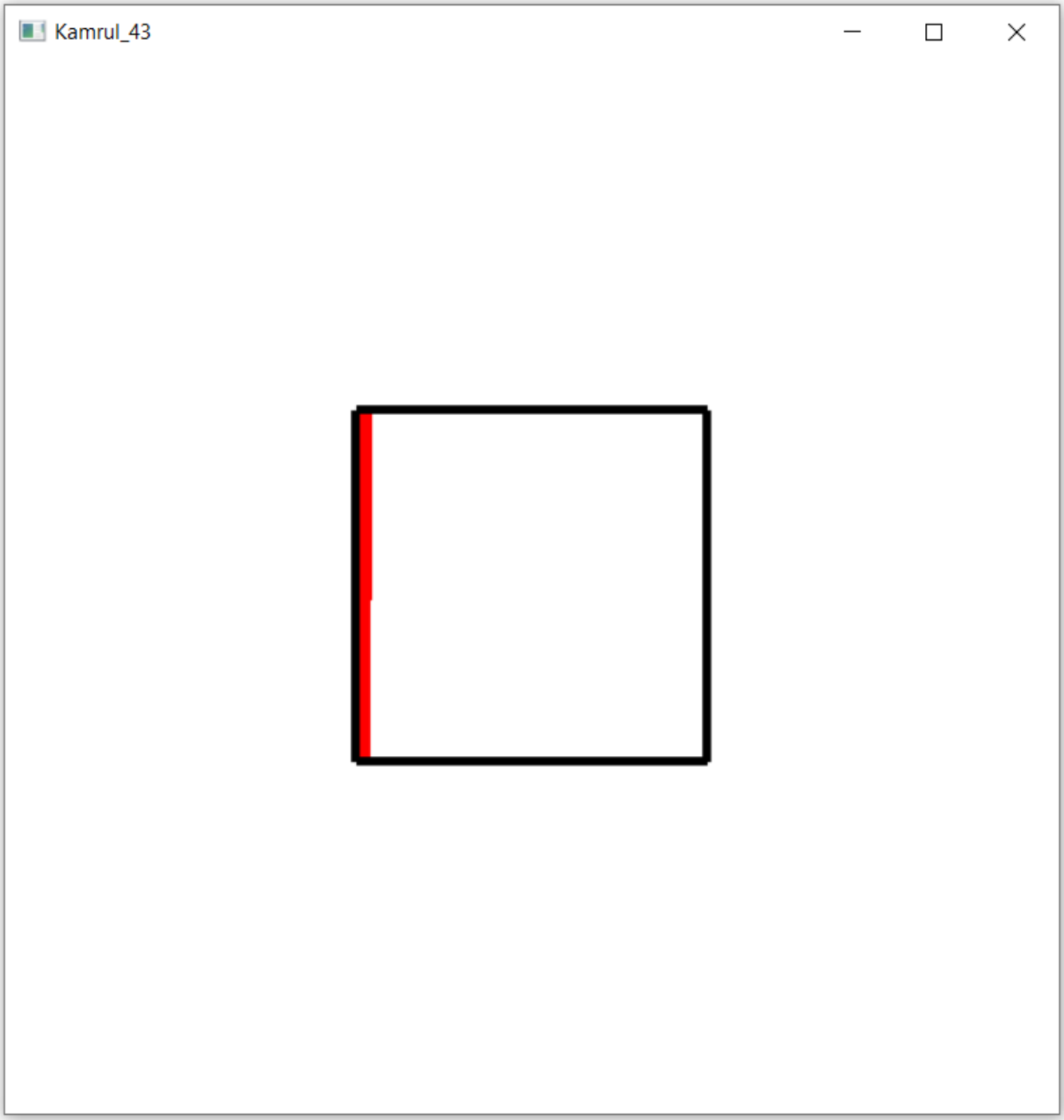
```python
# -------- Bresenham Circle --------
def drawCircle(cx, cy, r):
    points = []

    x = 0
    y = r
    d = 3 - 2 * r   # initial decision parameter

    while x <= y:
        # 8-way symmetry points
        pts = [
            (cx+x, cy+y), (cx-x, cy+y),
            (cx+x, cy-y), (cx-x, cy-y),
            (cx+y, cy+x), (cx-y, cy+x),
            (cx+y, cy-x), (cx-y, cy-x)
        ]
        points.extend(pts)

        if d < 0:
            d += 4*x + 6
        else:
            d += 4*(x-y) + 10
            y -= 1

        x += 1

    return points


# -------- Display Function --------
def show():
    glClear(GL_COLOR_BUFFER_BIT)   # clear screen

    # -------- DDA Line --------
    start = time.time()   # start time
    dda_points = DDA(100,100,300,300)   # get points
    dda_time = (time.time() - start) * 1000000   # convert to microsecond

    glPointSize(3.0)   # thickness
    glBegin(GL_POINTS)

    glColor3f(1,0,0)   # red color
    for x,y in dda_points:
        glVertex2i(x,y)   # draw pixel

    glEnd()
```

```python
    # -------- Delay --------
    time.sleep(3)   # wait 3 seconds

    # -------- Bresenham Line --------
    start = time.time()
    bres_points = BresenhamLine(300,100,100,300)
    bres_time = (time.time() - start) * 1000000

    glBegin(GL_POINTS)

    glColor3f(0,0,1)   # blue color
    for x,y in bres_points:
        glVertex2i(x,y)

    glEnd()

    # -------- Circle --------
    glPointSize(2.0)   # thinner
    glBegin(GL_POINTS)

    glColor3f(0,0,0)   # black
    circle = drawCircle(250,250,10)   # exact center
    for x,y in circle:
        glVertex2i(x,y)

    glEnd()

    # -------- Show Time --------
    glColor3f(0,0,0)

    drawText(50,560, f"DDA Time: {round(dda_time,2)} us")
    drawText(50,530, f"Bresenham Time: {round(bres_time,2)} us")

    glutSwapBuffers()   # update screen


# -------- Main Function --------
glutInit()   # initialize GLUT
glutInitDisplayMode(GLUT_RGBA | GLUT_DOUBLE)   # display mode
glutInitWindowSize(600,600)   # window size
glutCreateWindow(b"Kamrul_43")   # window title

glClearColor(1,1,1,1)   # background white
gluOrtho2D(0,600,0,600)   # coordinate system

glutDisplayFunc(show)   # set display function
glutMainLoop()   # run program
```

OUTPUT

Implementation of Stack Based Boundary Filling Algorithm.py

```python
from OpenGL.GL import *
from OpenGL.GLUT import *
from OpenGL.GLU import *

width, height = 600, 600

boundary_color = [0.0, 0.0, 0.0]   # boundary color (black)
fill_color = [1.0, 0.0, 0.0]       # fill color (red)

# -------- Compare Colors --------
def isSameColor(c1, c2):
    return all(abs(c1[i] - c2[i]) < 0.05 for i in range(3))

# -------- Get Pixel Color --------
def getPixel(x, y):
    glFlush()   # ensure drawing completed
    data = glReadPixels(x, y, 1, 1, GL_RGB, GL_FLOAT)
    return list(data[0][0])

# -------- Set Pixel --------
def setPixel(x, y, color):
    glColor3f(*color)
    glBegin(GL_POINTS)
    glVertex2i(x, y)
    glEnd()

# -------- Boundary Fill (Stack) --------
def boundaryFill(x, y):
    stack = [(x, y)]   # initialize stack

    while stack:
        x, y = stack.pop()

        current = getPixel(x, y)

        # check if pixel is not boundary and not already filled
        if not isSameColor(current, boundary_color) and not isSameColor(current,
fill_color):

            setPixel(x, y, fill_color)

            # push 4-direction neighbors
            stack.append((x+1, y))
            stack.append((x-1, y))
            stack.append((x, y+1))
            stack.append((x, y-1))
```

```python
# -------- Draw Boundary Shape --------
def drawShape():
    glColor3f(0,0,0)       # boundary color
    glLineWidth(5.0)       # thick boundary

    glBegin(GL_LINE_LOOP)

    glVertex2i(200,200)
    glVertex2i(400,200)
    glVertex2i(400,400)
    glVertex2i(200,400)

    glEnd()

# -------- Mouse Click --------
def mouse(button, state, x, y):
    if button == GLUT_LEFT_BUTTON and state == GLUT_DOWN:
        y = height - y   # coordinate conversion

        # fill only inside region
        if 200 < x < 400 and 200 < y < 400:
            boundaryFill(x, y)
            glFlush()


# -------- Display --------
def display():
    glClear(GL_COLOR_BUFFER_BIT)   # clear screen

    drawShape()   # draw boundary

    glFlush()

# -------- Main --------
glutInit()                              # initialize GLUT
glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB)
glutInitWindowSize(width, height)       # window size
glutCreateWindow(b"Kamrul_43")          # title

glClearColor(1,1,1,1)                   # background white
gluOrtho2D(0, width, 0, height)         # coordinate system

glutDisplayFunc(display)                # display function
glutMouseFunc(mouse)                    # mouse input

glutMainLoop()                          # run program
```

OUTPUT